

Syntax

Compose two or more automata in lock-step:

```
composition {
  synchronous pipeline {
    members [Producer, Consumer];
  }
}
```

The result `pipeline` is a new automaton you can reference in formulas and controllers just like any other.

Semantics

Both automata step **simultaneously** every cycle:

- **Shared labels** (same name in both automata) *must* fire together — they synchronize.
- **Non-shared labels** combine freely: each side picks one of its own transitions independently.
- A composed step is a pair of transitions, one from each member.

Example: `transfer` is shared; `produce`, `p_skip`, `consume`, `c_skip` are private.

State Space

The composed state space is the *product* of component state counts. E.g., $\text{Producer}(2) \times \text{Consumer}(2) = 4$ composed states. Keep components small to avoid explosion.

Skip Labels

Each automaton needs its own skip/idle label:

```
// Producer
transition Empty -> Empty
  on label p_skip;
transition DataReady -> DataReady
  on label p_skip;

// Consumer
transition Waiting -> Waiting
  on label c_skip;
transition Processing -> Processing
  on label c_skip;
```

Add a self-loop with the skip label on *every* state of the automaton.

Shared Idle Trap

Do **NOT** use the same idle label in both automata:

```
// WRONG --both use "idle"
transition A -> A on label idle; // in Automaton1
transition X -> X on label idle; // in Automaton2
```

Because `idle` is shared, both are forced to idle *together* — neither can act independently. Always use distinct names (`p_skip`, `c_skip`).

Formulas on Compositions

Use *structural* formulas (no state-name predicates) on composed systems:

```
// Safety over composition
formula safety_invariant {
  over pipeline;
  body = nu X. ([] X);
}

// No deadlocks: every state has
// at least one successor
formula no_deadlock {
  over pipeline;
  body = nu X. ((<> true) && ([] X));
}
```

State-name predicates (`Empty`, `Waiting`) only work on individual members, not on the composition.

Per-Component Properties

Check properties on individual automata separately:

```
formula producer_can_produce {
  over Producer;
  body = mu X. (DataReady || <> X);
}
formula consumer_processes {
  over Consumer;
  body = mu X. (Processing || <> X);
}
```

Complete Example

```
context sync_producer_consumer {
  automata {
    automaton Producer { ... }
    automaton Consumer { ... }
  }
  composition {
    synchronous pipeline {
      members [Producer, Consumer];
    }
  }
  mu_formulas { ... }
}
```

Further Reading

Milner (1989) *Communication and Concurrency* — CCS synchronous product. Hoare (1985) *Communicating Sequential Processes* — shared-event synchronization.